

Universidad Bernardo O'Higgins

Facultad de Ingeniería

Ingeniería de Informática

**Sistema de Alerta Inteligente con ESP32, Sensores
Múltiples y Comunicación MQTT en Tiempo Real**

Asignatura: Robótica y Tecnologías Emergentes [LAB1].

Profesor: Matías Hinrichsen.

Integrantes: Branco Merino.

Fecha: 05 de mayo del 2026.

Tabla de contenido

Sistema de Alerta Inteligente con ESP32, Sensores Múltiples y Comunicación MQTT en Tiempo Real	1
1. Introducción.....	1
2. Descripción del sistema	1
3. Implementación del circuito en Wokwi	2
4. Arquitectura del sistema.....	2
5. Lógica de funcionamiento.....	3
6. Comunicación MQTT.....	3
7. Descripción del firmware	4
8. Análisis de resultados.....	5
9. Limitaciones y mejoras futuras	5
10. Conclusión.....	5

1. Introducción

En situaciones de emergencia, muchas personas no pueden solicitar ayuda de forma inmediata debido a limitaciones físicas, falta de acceso a dispositivos o necesidad de actuar con discreción.

En este contexto, se propone un sistema de alerta automatizado capaz de activarse sin interacción directa, permitiendo generar una señal de auxilio de manera rápida.

El presente proyecto implementa una solución basada en un microcontrolador ESP32, utilizando sensores y comunicación mediante el protocolo MQTT. La simulación se realizó en la plataforma Wokwi, permitiendo validar el comportamiento del sistema en tiempo real sin necesidad de hardware físico.

2. Descripción del sistema

El sistema desarrollado consiste en un prototipo que permite detectar condiciones de riesgo mediante sensores y activar una alerta local (LED), además de enviar información en tiempo real a través del protocolo MQTT.

El sistema puede ser activado mediante una señal simulada, representando un escenario de activación por voz.

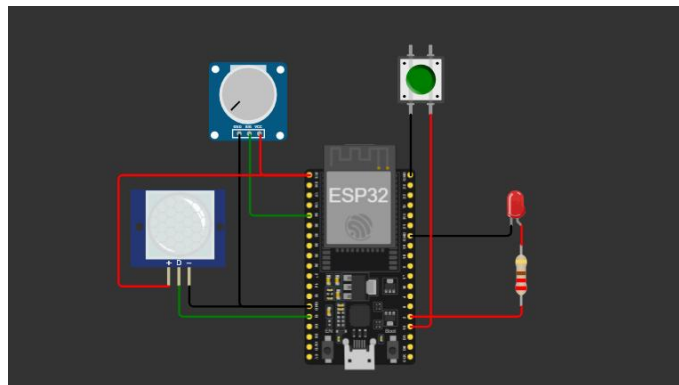
Además, permite el monitoreo remoto y control mediante mensajes enviados desde un cliente MQTT.

3. Implementación del circuito en Wokwi

El sistema fue implementado en la plataforma Wokwi utilizando un ESP32, integrando los siguientes componentes:

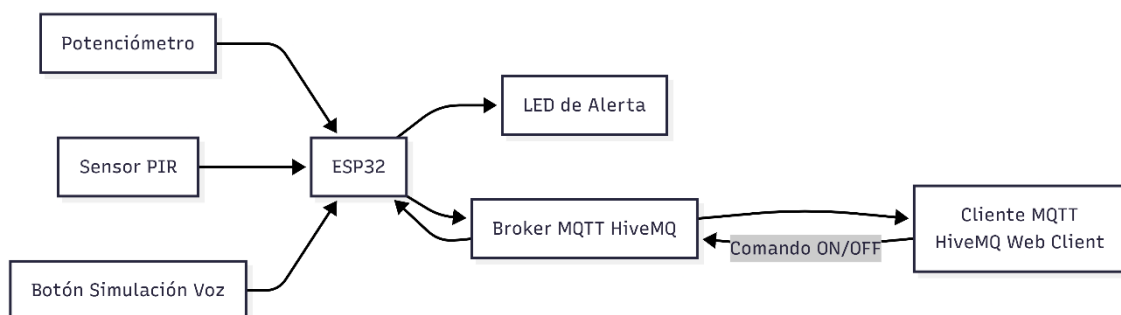
- Sensor PIR (detección de movimiento)
- Potenciómetro (entrada analógica variable)
- Botón (simulación de micrófono)
- LED (indicador de alerta) + Resistencia 220Ω (Protección del LED)

El funcionamiento se basa en una lógica de control que evalúa constantemente el estado de los sensores.



4. Arquitectura del sistema

El sistema sigue una arquitectura basada en IoT:



Descripción del flujo:

- Los sensores capturan información del entorno
- El ESP32 procesa los datos
- La información se envía al broker MQTT (HiveMQ)
- Un cliente MQTT permite visualizar y controlar el sistema

5. Lógica de funcionamiento

El sistema opera bajo tres estados principales:

- **Estado de reposo:** El sistema se mantiene monitoreando sensores sin activar alerta.
- **Estado de alerta:** se activa cuando:
 - El valor del potenciómetro supera un umbral
 - Se detecta movimiento mediante el sensor PIR
 - Se presiona el botón
- **Estado de control remoto:** el sistema responde a comandos MQTT, permitiendo activar o desactivar el LED de forma externa.

6. Comunicación MQTT

El sistema utiliza el protocolo MQTT mediante el broker público HiveMQ, permitiendo la comunicación en tiempo real.

Broker utilizado

- Broker: broker.hivemq.com
- Puerto: 1883

Topics implementados

Topic	Función
ubo/g1410/control	Permite enviar comandos al sistema.
ubo/g1410/sensor/pot	Publicación del potenciómetro.
ubo/g1410/sensor/mov	Publicación del sensor PIR.
ubo/g1410/sensor/alerta	Publicación del estado de alerta.
Ubo/g1410/sensor/boton	Publicación del estado del botón.

Funcionamiento de la comunicación

El ESP32 se conecta al broker MQTT y se suscribe al tópico de control. Posteriormente, publica información periódicamente utilizando distintos tópicos asociados a sensores y estados del sistema.

Cuando un cliente MQTT envía un mensaje de control:

1. El broker recibe el mensaje
2. El ESP32 recibe el tópico suscrito
3. Se ejecuta la función callback
4. El LED cambia de estado según el mensaje recibido

The screenshot displays a MQTT client interface with the following components:

- Connection:** A green dot indicates the client is "connected".
- Subscriptions:** A list showing a subscription to the topic "ubo/g1410/sensor/#" with a QoS of 2.
- Messages:** A list of received messages:
 - Topic: ubo/g1410/sensor/mov, QoS: 0, Message: Sin movimiento
 - Topic: ubo/g1410/sensor/alerta, QoS: 0, Message: Alerta no activada
 - Topic: ubo/g1410/sensor/boton, QoS: 0, Message: Boton no presionado
 - Topic: ubo/g1410/sensor/pot, QoS: 0, Message: 897
- Publish:** A form to send messages to the topic "ubo/g1410/control" with QoS 0. The message "ON" is entered.
- Hardware Diagram:** A schematic showing an ESP32 microcontroller connected to a potentiometer, a push button, and an LED.
- Terminal Output:** A log showing the received message: "[MQTT RECV] Topic: ubo/g1410/control | Msg: ON" and the resulting state: ">> LED encendido manualmente", followed by a list of sensor data: "----- ESTADO -----", "897", "Sin movimiento", "Boton no presionado", and "Alerta no activada".

7. Descripción del firmware

El sistema fue desarrollado utilizando el entorno Arduino Framework en PlatformIO.

Las principales características implementadas son:

Lectura periódica de sensores, uso de la función **millis()** para evitar bloqueos por **delay()**, publicación continua de datos MQTT, suscripción a tópicos de control, reconexión automática al bróker MQTT y mensajes de depuración mediante Serial monitor.

La lógica principal se ejecuta dentro del **loop()**, donde se verifican conexiones MQTT, lectura de sensores y evaluación continua de condiciones de alerta.

8. Análisis de resultados

Las pruebas realizadas permitieron verificar el correcto funcionamiento del sistema, comprobando la publicación de datos en tiempo real, la recepción de comandos MQTT, la activación correcta del LED y la detección de eventos mediante el sensor PIR y el potenciómetro. Además, el protocolo MQTT demostró ser una solución adecuada para aplicaciones IoT debido a su baja latencia y eficiencia en la comunicación entre dispositivos y clientes remotos.

9. Limitaciones y mejoras futuras

Durante el desarrollo se identificaron distintas limitaciones asociadas a la simulación.

Limitaciones:

Wokwi no permite implementar reconocimiento de voz real, por lo que se utilizó un botón como simulación de activación por voz. Además, las pruebas fueron realizadas únicamente en un entorno virtual.

Mejoras futuras:

Integración de reconocimiento de voz real, notificaciones móviles, una interfaz web de monitoreo, almacenamiento de eventos y sensores adicionales para ampliar las capacidades del sistema.

10. Conclusión

El sistema desarrollado cumple con la propuesta inicial, permitiendo detectar condiciones distintas y generar alertas automáticas mediante sensores y comunicación MQTT. El proyecto demuestra la aplicación de tecnologías IoT utilizando ESP32 para desarrollar sistemas de alerta automatizados, soluciones de monitoreo y control en tiempo real. La integración de sensores y MQTT permitió desarrollar un sistema funcional en tiempo real, escalable y adaptable aportando al desarrollo de herramientas tecnológicas en el área de seguridad y monitoreo.